

Predicción de la estructura secundaria de proteínas mediante k-NN

Joaquín Villegas

Tabla de contenidos

1	Predicción de la estructura secundaria de proteínas mediante k-NN	1
1.1	Introducción	1
1.2	Datos	2
1.3	Métodos	3
1.3.1	Algoritmo	3
1.3.2	Diseño experimental	3
1.3.3	Métricas de evaluación	3
1.4	Resultados	4
1.6	Discusión	5
1.7	Conclusiones	5
1.8	Referencias	5

1 Predicción de la estructura secundaria de proteínas mediante k-NN

1.1 Introducción

La predicción de la estructura secundaria de proteínas es un problema que a día de hoy sigue sin resolverse. La estructura de las proteínas está estrechamente relacionada con su función biológica y conocerla de forma precisa nos permite acercarnos a entender el funcionamiento de la misma. Así como ser capaces de modular su acción, ya sea inhibiéndola o estimulándola, siendo este uno de los objetivos principales del diseño de fármacos (drug design) y optimización de enzimas convirtiendo así el estudio de la estructura proteica en un área de estudio tan relevante en la industria farmacológica y biotecnológica.

Existen varias herramientas para el estudio de la estructura de las proteínas, uno de los enfoques más prometedores a día de hoy está basado en aprendizaje automático (machine learning). Siendo un hito el lanzamiento de AlphaFold, una plataforma que predice, y da índices de

fiabilidad por aminoácidos, la estructura 3D de proteínas a partir de su estructura primaria (la secuencia de aminoácidos).

La herramienta de aprendizaje automático que hemos utilizado en este estudio para abordar la predicción de la estructura secundaria a partir de la secuencia de aminoácidos es el algoritmo k-NN, que clasificará la estructura proteica secundaria de nuevas secuencias tomando como referencia las k instancias que se aproximen con mayor similitud a la secuencia de aminoácidos nueva. A diferencia de otros algoritmos de machine learning este modelo no aprende nada por eso es llamado lazy learner. A pesar de la simplicidad del modelo sigue siendo muy utilizado a día de hoy.

1.2 Datos

Los datos han sido obtenidos del repositorio de machine learning UC Irvine, Molecular Biology (Protein Secondary Structure) y provienen del repositorio connectionist bench del CMU. Los datos de las secuencias se presentan en ventanas deslizantes, teniendo cada instancia 17 ventanas con aminoácidos.

Deberemos realizar una transformación categórica de los aminoácidos de tal forma que se genere una matriz de datos de 0 y 1 donde cada instancia será una matriz de 17x20 generando así 340 columnas (Table 1). Antes de utilizar el one-hot encoding method (generando variables dummy o variables indicadoras) separamos la variable respuesta/independiente de la variables dependientes y usamos la libreria caret.

Factorizamos los aminoácidos por si existe algún aminoácido el cual no aparece en las secuencias obtenidas evitar que el one-hot encoding ignore ese aminoácido/variable.

y

```
coil Bsheat Ahelix
5557   1935   2508
```

Table 1: Resultado de los datos tras el One-hot encode

V1.AV1.RV1.NV1.DV1.CV1.EV1.QV1.GV1.HV1.I V1.LV1.KV1.MV1.FV1.PV1.SV1.TV1.WV1.YV1.V																			
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0

1.3 Métodos

1.3.1 Algoritmo

El método k-nn es un método que utiliza la similitud entre instancias para clasificar esa instancia en una clase determinada. El algoritmo que vamos a utilizar trata las instancias como coordenadas en un espacio de dos dimensiones. La distancia entre dos puntos o dos instancias vienen dada por una función de distancia, la más usada y la utilizada en este ensayo es la distancia euclídeana.

Es necesario determinar el número de puntos (k) a tener en cuenta para clasificar una nueva instancia. La elección de un k óptimo es un punto clave para determinar la calidad de nuestro modelo, suponiendo un reto hallar el equilibrio entre sobreajustar (generalmente cuando la k es muy pequeña) o generar un modelo que generalice demasiado (con una k demasiado grande) (bias-variance tradeoff).

1.3.2 Diseño experimental

Particionamos los datos en trainx , trainy , testx , testy mediante la función `createDataPartition` del paquete `caret` indicando que el 70% de los datos sean el train y el 30% el test . La elección de estos parámetros busca proporcionar suficientes datos para que el modelo aprenda los patrones y una porcentaje representativo para validar la capacidad del modelo. Para generar nuestro modelo también usaremos la herramienta “cross validation” que nos permite dividir nuestros datos de entrenamiento en k pliegues (10 en nuestro caso, el estándar) y de tal forma que el modelo aprende con $k-1$ pliegues y lo enfrenta al pliegue restante. El proceso se hace de forma iterativa rotando sucesivamente los $k-1$ pliegues y el pliegue restante.

Hemos probado tres valores diferentes de k (1, 3 y 5) puesto que el algoritmo knn suele alcanzar un equilibrio óptimo a valores más pequeños de k , obteniendo un rendimiento mayor a $k=1$. Para evitar que el modelo obtenga valores igualados de dos clases diferentes se tiende a usar valores impares.

1.3.3 Métricas de evaluación

Con el fin de evaluar el modelo y calcular su rendimiento se empleó una matriz de confusión la cual permite desglosar los valores reales (filas) frente a los valores predichos (columnas) Table 2). A partir de esta matriz se derivan métricas específicas de clase como son la sensibilidad, especificidad, precisión y recall para entender si nuestro modelo sufre más (comete más errores) a la hora de clasificar instancias en ciertas clases (Table 3). Además de métricas globales como el ratio de aciertos del modelo (accuracy) y el ratio de error que permiten evaluar el modelo de forma general (Table 4).

También se utilizó el estadístico Kappa, útil cuando las clases están desbalanceadas puesto que evalúa la eficacia del modelo restando el efecto del azar (Table 4).

Table 2: Matriz de confusiónn

	coil	Bsheet	Ahelix
coil	1383	155	155
Bsheet	132	378	48
Ahelix	152	47	549

Complementariamente, se generaron diagramas ROC para visualizar la relación entre la tasa de verdaderos positivos y falsos positivos a distintos umbrales (Figure 1) El rendimiento de los modelos puede medirse mediante el estadístico AUC (area under the ROC curve) con rangos desde 0.5 (sin capacidad de predicción) a 1 (clasificador perfecto) (Table 4).

1.4 Resultados

Table 3: Resumen de métricas por clase del Modelo k-NN

	Class: coil	Class: Bsheet	Class: Ahelix
Sensitivity	0.8296341	0.6517241	0.7300532
Specificity	0.7672673	0.9255891	0.9114375
Pos Pred Value	0.8168931	0.6774194	0.7339572
Neg Pred Value	0.7825421	0.9172470	0.9098179
Precision	0.8168931	0.6774194	0.7339572
Recall	0.8296341	0.6517241	0.7300532

Table 4: Resumen de Estadísticos del Modelo k-NN

Estadísticos	Valores
Accuracy	0.7702568
Kappa	0.6090704
AUC Multiclase	0.8492870

coil.Bsheet	coil.Ahelix	Bsheet.Ahelix	Freq
0.837088099621455	0.860405445506643	0.840471386647102	1

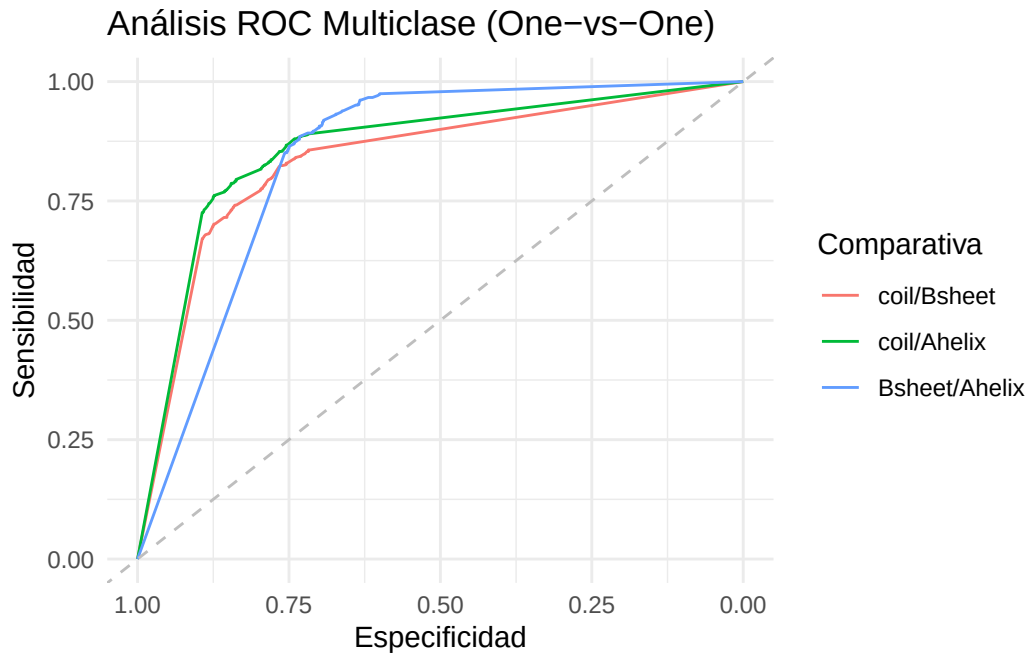


Figure 1: Analisis ROC Multiclase (1vs1)

1.5

1.6 Discusión

1.7 Conclusiones

1.8 Referencias